

Ontario Tech University

ENGR 5785G: Real-Time Data Analytics for IoT

Assignment 4**Time Series Analysis Report**

Student Name	Student ID
Vitor Brandao Raposo	101011969

1. Introduction

Short-term traffic-speed forecasting is the entry point for most intelligent transportation systems. Knowing the speed 15 minutes ahead is enough lead time for route guidance, ramp metering, and incident alerts. This assignment builds that workflow on a single freeway sensor from the public PeMSD7 dataset, then compares a classical statistical model against a tree-based machine-learning model, and uses the better model's residuals to flag unusual traffic moments.

2. Data and Preprocessing

The PeMSD7_V_228.csv file from the STGCN benchmark contains 12,672 rows of 5-minute speed measurements across 228 freeway sensors — about 44 days of data from 2012-05-01 to 2012-06-13 (a synthetic timestamp index, since the raw CSV does not include timestamps). Sensor 0 is used as the target series.

stat	mean	median	std	min	max	missing
value	52.62	59.70	19.44	3.30	80.90	0

The median is well above the mean — speed sits near the free-flow ceiling most of the day and drops sharply during the two daily commute peaks, which pulls the mean down. There are no missing values in this sensor, so the `interpolate().ffill().bfill()` step is precautionary.

Engineered features (16 columns total):

- Lags at 1, 2, 3, 6, and 12 steps (5 min to 1 h of recent history)

- Rolling mean and standard deviation at windows of 6, 12, and 24 steps, all shifted by one so the feature at time t never includes t itself (no leakage)
- Calendar features: hour, dayofweek, is_weekend

The forecast target is speed 15 minutes ahead (shift(-3)). After dropping rows with NaN from the windowing, the feature matrix is $12,645 \times 16$.

3. Methods

Three approaches are compared on chronologically held-out test windows.

- Persistence baseline. Predict $y_{\{t+15\text{min}\}} = y_t$. The natural reference point for short-horizon traffic data.
- SARIMA(1,1,1)(0,1,1,24) on hourly data. A SARIMA fit on the native 5-minute series needs a seasonal period of 288 (24×12). The state-space Kalman filter for SARIMAX scales with the seasonal length and the fit becomes impractical at $m=288$. The series was therefore resampled to hourly means for SARIMA only ($m=24$), trained on the last 21 days (504 hourly points), and used to forecast the next 24 hours.
- Random Forest ($n_{\text{estimators}}=100$, $\text{max_depth}=12$) on the engineered feature matrix at the native 5-minute resolution. Chronological 80/20 split — train = 10,116 rows, test = 2,529 rows (about 9 days).

Anomalies are detected from Random Forest residuals (actual minus predicted) on the test window. A timestamp is flagged when $|\text{residual}|$ exceeds the 99th percentile of absolute residuals — a simple, interpretable threshold that adapts to the residual scale without assuming a distribution..

4. Results

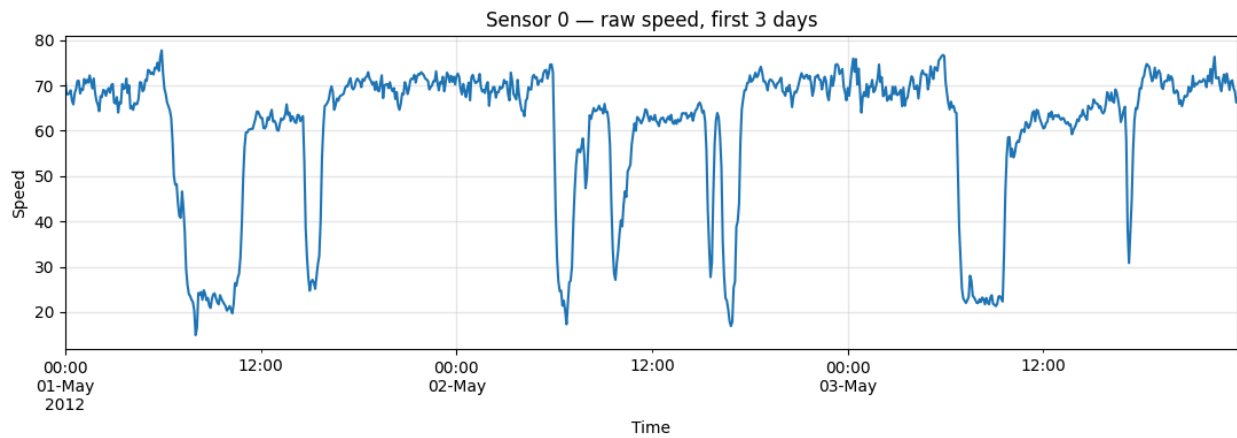
Model	MAE	RMSE	MAPE (%)
Persistence (5-min, 15-min ahead)	4.657	7.559	11.34
Random Forest (5-min, 15-min ahead)	4.441	6.934	11.09
SARIMA (hourly, 24-h ahead)	7.468	8.944	n/a

The SARIMA row is reported on a different time scale and forecast horizon (one hour vs. five minutes, one day vs. 15 minutes ahead), so it is not directly comparable to the other two rows. It

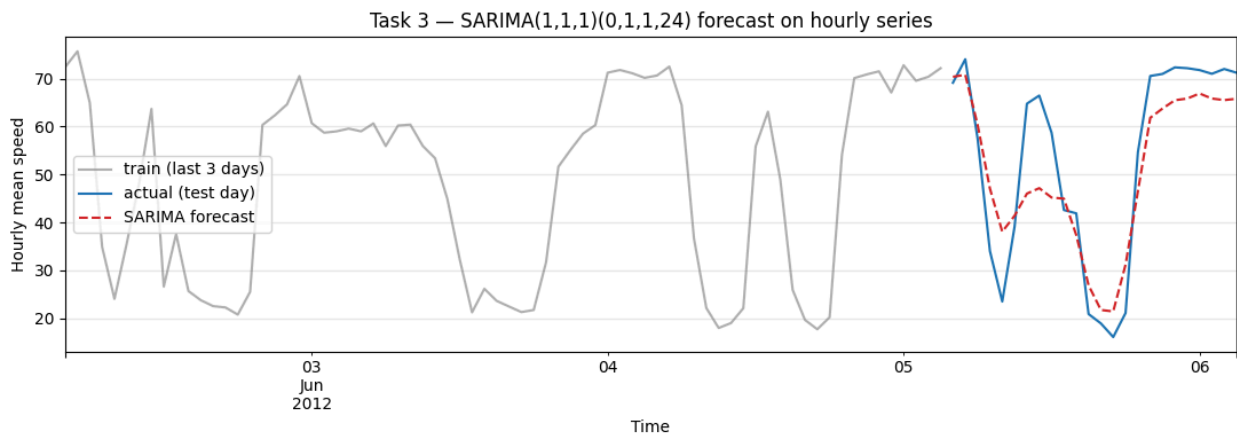
is included because the assignment requires a classical model. The fair head-to-head is Persistence vs. Random Forest, both at 5-minute resolution.

Random Forest beats persistence on every metric (~5% better MAE, ~8% better RMSE). The anomaly detector flagged 26 points in the test window with a threshold of $|\text{residual}| > 24.78$ mph (the 99th percentile of absolute residuals). The 10 largest residuals all fall during evening-rush hours (17:30–19:30) or late-morning congestion windows (11:00–14:30), which is the hardest part of the day for a lag-driven model.

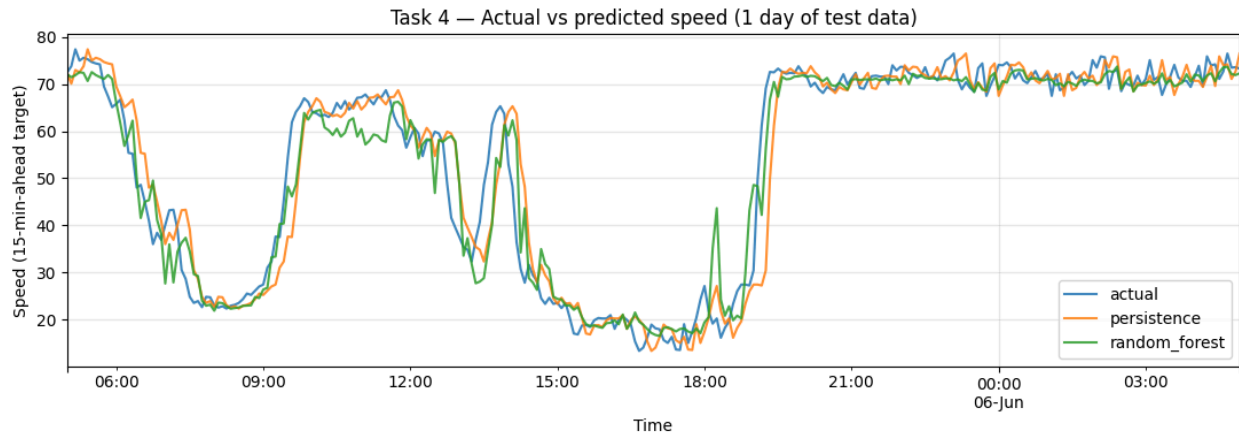
Here are the outputs of all the analysis made:



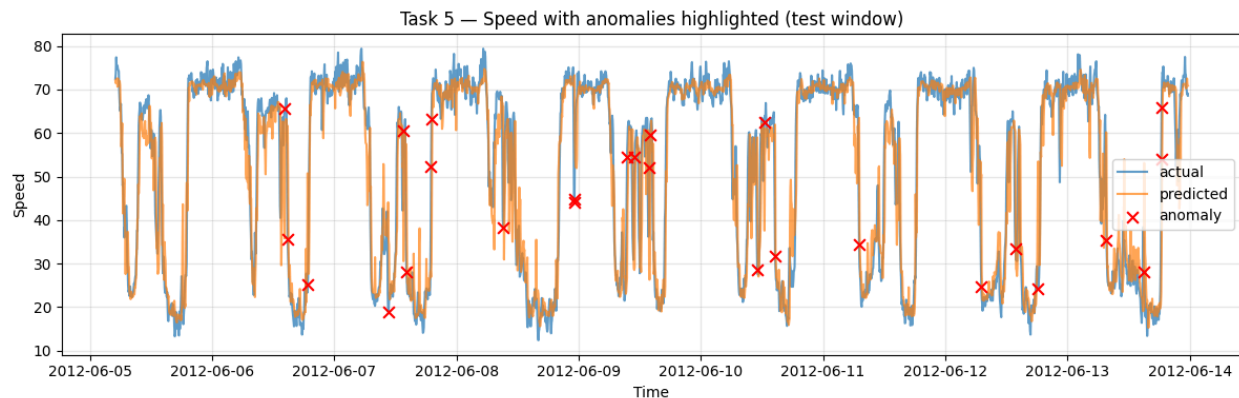
Three days of raw speeds showing the twice-daily commute drops.



SARIMA 24-hour forecast vs. actual hourly speed.



One day of test data with the persistence baseline and the Random Forest prediction overlaid on the truth.



The test window with anomalies marked in red.

5. Discussion

Best mode: Random Forest is the best of the three at the assigned 15-minute horizon. It combines the most recent lag with rolling statistics and time-of-day context, so it reacts to both short-term momentum and the daily pattern. SARIMA on hourly data captures the daily cycle but cannot help on a 15-minute target because the hourly resampling destroys the fine-grained variation a 15-minute forecaster needs.

Limitations:

1. Resampled SARIMA. SARIMA was forced to run on hourly means because the $m=288$ state space made the runtime infeasible on my laptop. This makes the SARIMA vs. Random Forest comparison apples-to-oranges. A model designed for long seasonal periods (Holt-Winters with seasonal damping, STL decomposition + ARMA on the residual, or Prophet) would let us compare classical and ML at the same horizon.

2. Single sensor only. The model ignores spatial information from neighbouring sensors. Real freeway speed depends on what is happening upstream and downstream, and the PeMSD7 distance matrix would let a graph model exploit that — out of scope here.

Real-time deployment idea: The natural extension is to stream incoming sensor readings through a Kafka topic and run an incremental learner (for example, a `river.linear_model.LinearRegression`) that updates its weights on each new sample. The Random Forest in this notebook is trained once on a fixed window, so its accuracy will drift over time; an online learner combined with the same engineered features would keep up with seasonal drift, road work, or sensor recalibration without a retraining job.

6. References

1. Caltrans Performance Measurement System (PeMS).
<https://dot.ca.gov/programs/traffic-operations/mpr/pems-source>
2. Yu, B., Yin, H., Zhu, Z. (2018). STGCN IJCAI-18 public benchmark dataset.
https://github.com/VeritasYin/STGCN_IJCAI-18
3. Keras traffic forecasting example using PeMSD7.
https://keras.io/examples/timeseries/timeseries_traffic_forecasting/
4. statsmodels SARIMAX documentation.
<https://www.statsmodels.org/stable/generated/statsmodels.tsa.statespace.sarimax.SARIMAX.html>
5. scikit-learn RandomForestRegressor documentation.
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>